

Task 2: Ranking

keyword_match assigns 1 for any shared query token and otherwise 0, it does not distinguish term rarity, repetition or document length. I implemented BM25 using index-stored df/tf, with IDF weighting, saturated term frequency ($k1$) and document-length normalisation (b). I set the baseline BM25 to use $k1=1.2$, $b=0.75$ (as per lecture suggestion)

Method	Settings	Dev Pearson
keyword_match	Binary token overlap	0.000
BM25 (base)	$k1 = 1.2, b = 0.75$	0.473
BM25 (finet-tuned/default)	$k1 = 3, b = 0.50$	0.492
BM25 (extreme values)	$k1 = 0.0, b = 0.00$	0.406
BM25 (high $k1$, $k1 = 5+$)	$k1 = 5.0, b = 0.50$	0.494

BM25 improved Pearson over keyword_match because it considers more than just whether a document shares at least one query term. It also considers how rare the matched terms are, how often they appear in the document, and the document length. I first tested the standard BM25 settings as per the lecture ($k1=1.2$, $b=0.75$), which gave a **Pearson score of 0.473**.

When **$k1$ was fixed at 1.2**, varying b showed that **$b=0.5$ performed best**, although the differences were small. I then fixed **$b=0.5$** and varied $k1$; performance improved up to around **$k1=3$** , reaching **0.492**, before plateauing at 3.0+. Higher values such as $k1=5$ only improved slightly to 0.494, so I kept $k1=3$ because the extra gain was negligible. Pearson measures how closely the ranking scores follow the graded relevance scores, so the higher result suggests BM25 was better at assigning larger scores to more relevant documents.

Task 3: System Optimisation

The provided starter achieved **MAP 0.2351**. Applying consistent document and natural-language query preprocessing increased **MAP to 0.2867**. Query tokenisation was very useful for punctuation: e.g. N09 contains “flutter,” which whitespace splitting treats as a different token from “flutter”.

Method	Description	MAP score
run_no_optimisation.json	the provided starter	0.2351
run_preprocessing.json	doc_cleaning + query_cleaning	0.2867
run_default.json	using prf top_m = 20 and top_docs = 3	0.3292

I then added pseudo-relevance feedback (PRF). My algorithm initially does a BM25 search, assumes the **top 3 documents** are relevant, extracts the **20 highest TF-IDF terms** from them, expands the query and does a second search. This **increased MAP to 0.3292**. For query N09, preprocessing retrieved 4/7 relevant documents in the top 10 and with PRF retrieved 6/7, with relevant documents at ranks 1 - 5 and 9. PRF sometimes caused query drift, but the gains outweighed these losses as seen in queries such as N09 and N03, therefore **preprocessing + PRF was selected as default**.